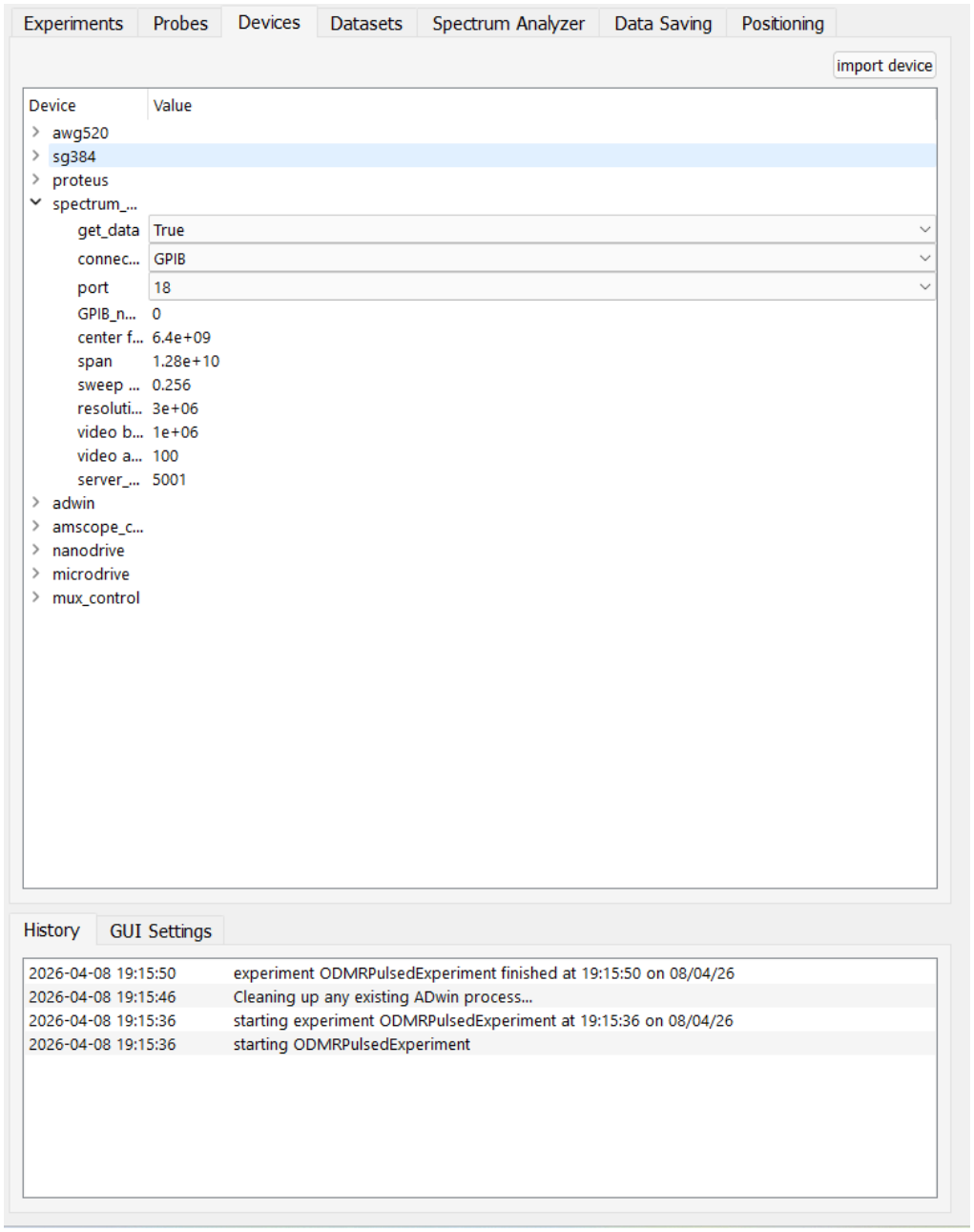
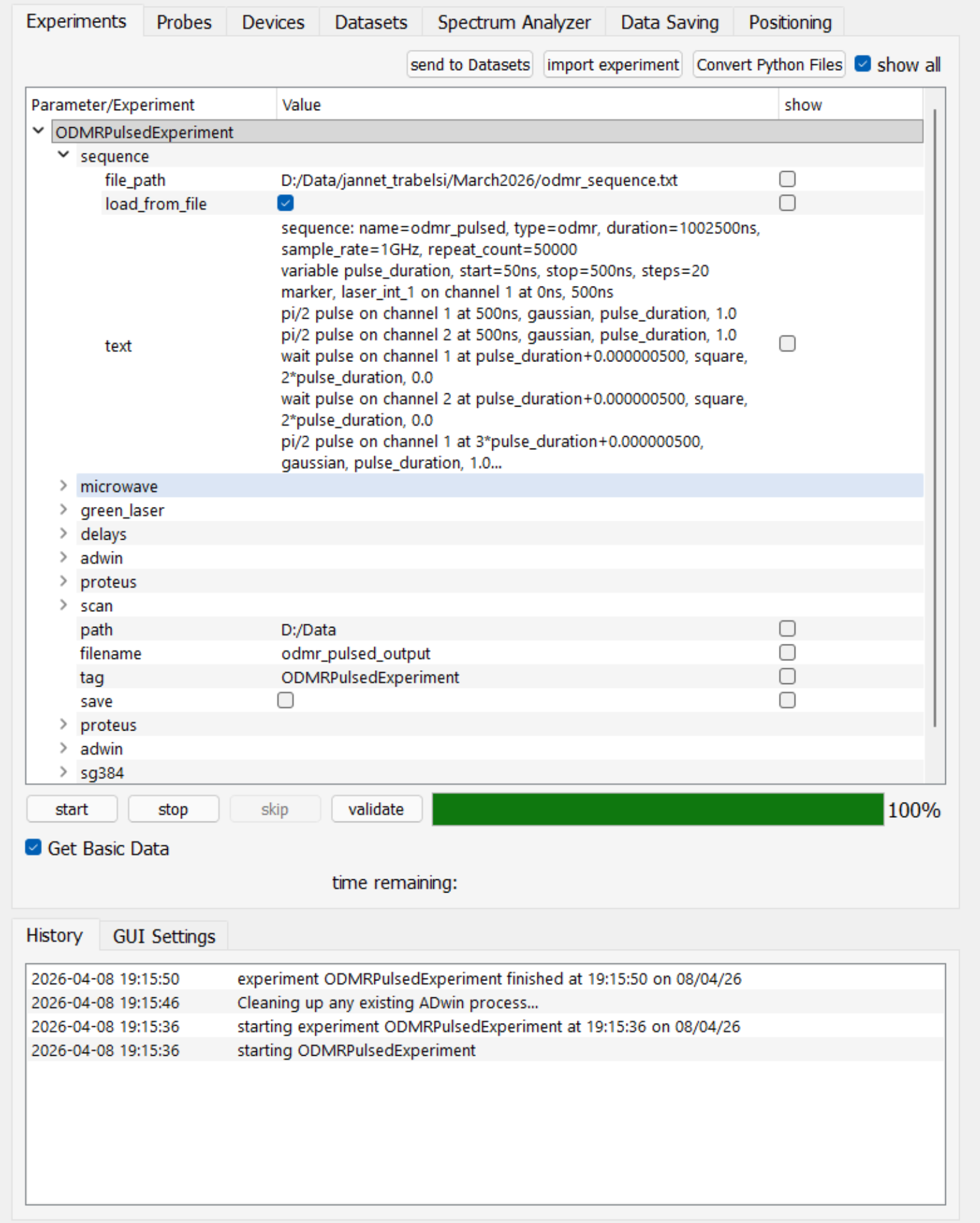
Users:

To save data from different devices, we do the following steps:

1. Go to devices tab and select devices that you want to include: select true for the get_data parameter (please note that if there is a device that is shared with the experiment, the experiment will override any parameter selections, unless you programmed it in an interactive way. For example, the confocal point experiment is interactive since you can change the drives positions. However, the pulsed ODMR experiment is not interactive. Everything is set in the sequence description before you click “start”):



2. Go to the experiment tab and click the “Get Basic Data” checkbox:



3. Start experiment

4. To check if this worked properly, go to the devices tab and click on your file and click read data:

Experiments | Probes | Devices | Datasets | Spectrum Analyzer | Data Saving | Positioning

D:\Data

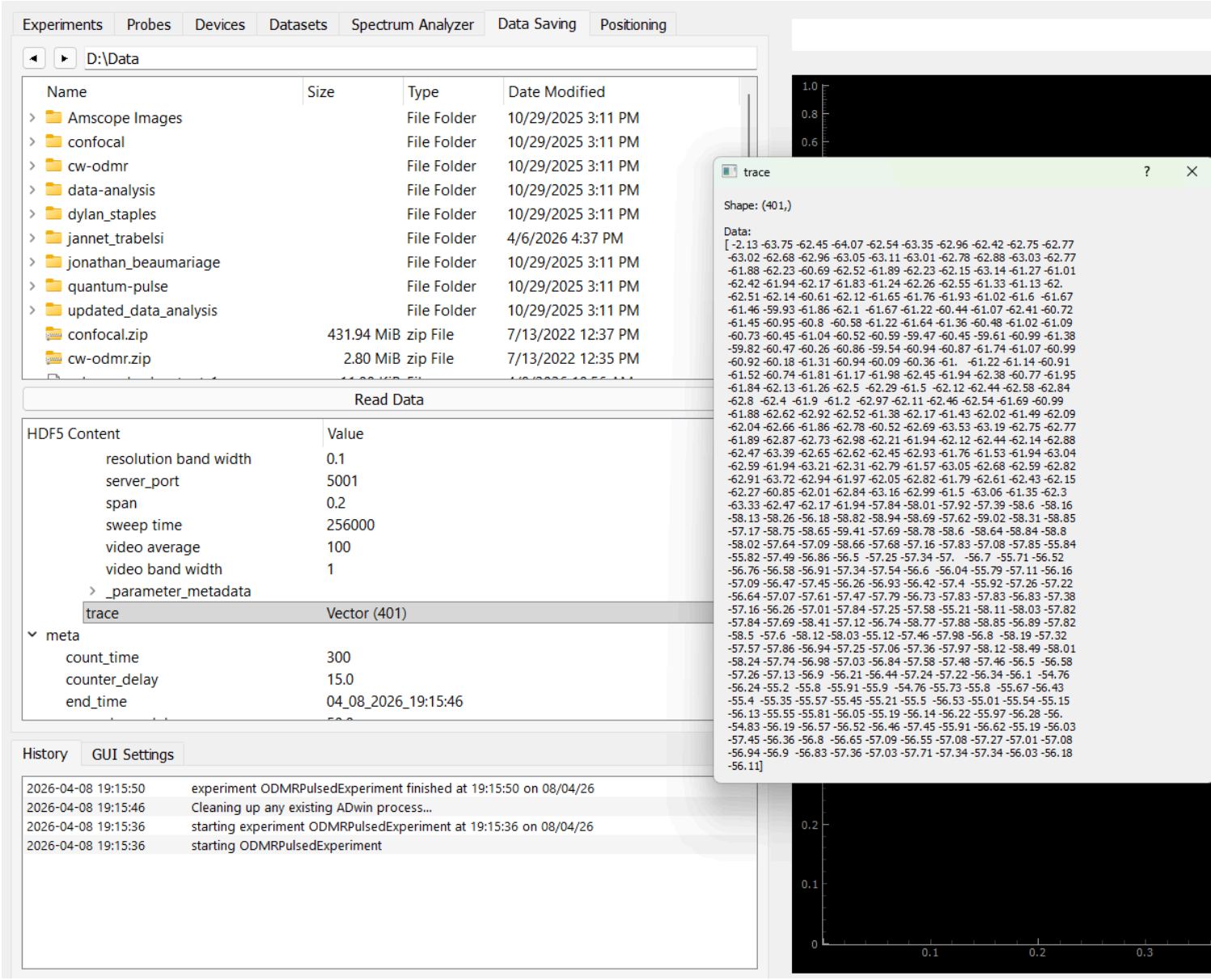
Name	Size	Type	Date Modified
jannet_trabelsi		File Folder	4/8/2026 8:31 PM
jonathan_beaumariage		File Folder	10/29/2025 3:11 PM
quantum-pulse		File Folder	10/29/2025 3:11 PM
updated_data_analysis		File Folder	10/29/2025 3:11 PM
confocal.zip	431.94 MiB	zip File	7/13/2022 12:37 PM
cw-odmr.zip	2.80 MiB	zip File	7/13/2022 12:35 PM
odmr_pulsed_output_04_09_2026...	9.24 KiB	h5 File	4/9/2026 3:52 PM
odmr_pulsed_output_04_09_2026...	22.26 KiB	h5 File	4/9/2026 4:46 PM
odmr_pulsed_output_04_09_2026...	31.38 KiB	h5 File	4/9/2026 4:54 PM
odmr_pulsed_output_04_09_2026...	31.38 KiB	h5 File	4/9/2026 5:00 PM
odmr_pulsed_output_04_09_2026...	48 bytes	h5 File	4/9/2026 5:03 PM

Read Data

HDF5 Content	Value
basic_data_devices	
spectrum_analyzer	
params	
GPIB_num	0
center frequency	300
connection_type	GPIB
get_data	True
port	18
resolution band width	0.1
server_port	5001
span	0.2
sweep time	256000

History | GUI Settings

2026-04-09 17:03:56	experiment ODMRPulsedExperiment finished at 17:03:56 on 09/04/26
2026-04-09 17:03:51	Cleaning up any existing ADwin process...
2026-04-09 17:03:41	starting experiment ODMRPulsedExperiment at 17:03:41 on 09/04/26
2026-04-09 17:03:41	starting ODMRPulsedExperiment



5. Now you see that the spectrum analyzer, which is not part of the odmr experiment, is now included. Its parameters and its data “trace” is also included

Programmers:

Main window file: the main window sends to the experiment whether Get Basic Data is clicked or not AND the devices selected true for get_data in the devices tab:

```python
class MainWindow(QMainWindow, Ui_MainWindow):  Gurudev Dutt +6 *
    def btn_clicked(self):  B103-PC +4 *
        def start_button():  B103-PC +2 *
            #         self.expanded_items.append(someitem.name)
            self.experiment_start_time = datetime.datetime.now()

            if item is not None:
                # get experiment and update settings from tree
                self.running_item = item
                experiment, path_to_experiment, experiment_item = item.get_experiment()
                if self.getbasicdatacheckBox.isChecked():
                    checked_devices = []
                    print("The checkbox is CHECKED.")
                    for device_name, device_obj in self.devices.items():
                        print(device_name)
                        if device_obj.settings['get_data'] == True:
                            print(f"device {device_name}'s data is included")
                            checked_devices.append(device_obj)
                        else:
                            print(f"device {device_name}'s data is NOT included")
                    experiment.get_checked_devices(checked_devices)
                else:
                    print("The checkbox is UNCHECKED.")

                gui_logger.info(f"Starting experiment: {experiment.name}")
```

Parent experiment:

```python
def get_checked_devices(self, devices):  1 usage (1 dynamic)  new *
    self.checked_devices = devices

def save_hdf_data(self, structure_to_save: MyStruct):  1 usage  new *
    """
    filename is the selected path + filename from parameters
    structure_to_save is a MyStruct object that has structure_to_save.data, structure_to_save.meta and other stuff as wish, but now it will have structure_to_save.devices
    this class should not be overridden. This should get called by save_hdf5 in the subclass to ensure that when the Get Basic Data button gets pressed, the selected devices for
    get_data get their data collected. For more info, please refer to github: https://github.com/duttlab-sys/pittqlabsys-single-NV/tree/main/docs/guides/development/data_saving_documentation.pdf
    """
    base_path = Path(self.settings["path"])
    base_path.mkdir(parents=True, exist_ok=True)

    file_name = Path(self.settings["filename"]).stem
    time_now = datetime.datetime.now().strftime("%m_%d_%Y_%H_%M_%S")

    filename = base_path / f"{file_name}_{time_now}.h5"
    if self.checked_devices is None:
        print("no devices seleted to get basic data")
        return
    else:
        structure_to_save.basic_data_devices = MyStruct()
        for device in self.checked_devices:
            if device not in self.devices:
                dev_name = device.name
                setattr(structure_to_save.basic_data_devices, dev_name, device.get_data())

        save_data(filename, structure_to_save)

def save_hdf5(self):  new *
    """subclasses need to define their own data then call save_hdf_data(structure_to_save: MyStruct), this way, the Get Basic Data button gets to save data from selected external devices: for more
    raise NotImplementedError("Subclasses must implement this method: every experiment is supposed to save data using hdf5: please refer to save_hdf5 method implemented in odmr_pulsed file")
```

If the get basic data CHECKBOX is checked, the parent experiment gets the selected devices from main window as shown above, checks if the devices are not used in the experiment. Those devices get their **get_data**() function called and then it saves the data in an hdf5 file using the **save_data** function implemented in the struct_hdf5 file, here is an example:

The **parameter_to_mystruct** function is implemented in the struct_hdf5 file to save any parameters of any devices that are children of the Device class. If the only data that needs to be saved is parameters, you do not need to add anything to your device. Here is the implemented code:

Parent Device class:

```
def get_data(self):  new *
    """ data saving: any experiment that clicks Get Basic Data button can have the devices selected True in the devices tab for get_data saved along with the experiment data.
    If your device data that needs to be saved is only the parameters, please refrain from overriding the function. However, if you need to add more data, please override this function
    For example, please check Agilent8596E get_data function that overrides this function and adds self.data.trace. if you have a camera, you can have self.data.image and override this
    function to share that data
    For more info, please refer to github: https://github.com/duttlab-sys/pittqlabsys-single-NV/tree/main/docs/guides/development/data_saving_documentation.pdf"""
    self.data = MyStruct()
    self.data.params = parameter_to_mystruct(self.settings)
    return self.data
```

However, if any data needs to be added, please override this function as shown below:

Agilent8596E class as an example:

```
def get_data(self):  new *
    """ this is overriding the function in the parent device class to include self.data.trace as it is not present in the parameters"""
    self.data = MyStruct()
    self.data.params = parameter_to_mystruct(self.settings)
    self.data.trace = np.array(self.read_probes("trace AP"))
    return self.data
```

Child experiment:

This experiment defines the special data that needs to be saved in its custom save_hdf5 function and then calls the save_hdf_data that is present in the parent then the parent experiment takes care of saving the data of the devices that are not part of the experiment (the child experiment does not know about those devices)

```python
def save_hdf5(self):  1 usage  duttlab-sys *
    """this function defines its custom data and metadata to be saved and then calls the
    save_hdf_data function that is in the parent Experiment class, which adds the external
    devices in case you ever check the Get Basic Data checkbox in the GUI"""
    structure_to_save = MyStruct()
    signal_counts = self.data["signal_counts"]
    reference_counts = self.data["reference_counts"]
    total_counts = self.data["total_counts"]
    structure_to_save.data = MyStruct(
        signal_counts=signal_counts,
        reference_counts = reference_counts,
        total_counts = total_counts
    )
    structure_to_save.meta = MyStruct(
        count_time = self.count_time,
        counter_delay = self.counter_delay,
        end_time = self.e_t,
        frequency_range = self.data['frequencies'],
        green_laser_delay = self.green_laser_delay,
        green_laser_power = self.green_laser_power,
        green_laser_wavelength = self.green_laser_wavelength,
        iq_delay = self.iq_delay,
        microwave_power = self.microwave_power,
        mw_delay = self.mw_delay,
        number_of_iterations = self.number_of_iterations,
        proteus_response_delay = self.proteus_response_delay,
        repeat_count = self.repeat_count,
        reset_time = self.reset_time,
        sampling_rate = self.sampling_rate,
        sequence_text=self.sequence_text,
        sequence_duration = self.sequence_duration,
        start_time = self.s_t,
        success = self.data["success"],
        trigger_delay = self.trigger_delay
        )
    structure_to_save.devices = self.devices
    self.save_hdf_data(structure_to_save)
```

So future programmers only need to override **get_data()** function to their **devices** files in case they need more data other than parameters and override the **save_hdf5** in their **experiment** files, making sure that they call the **save_hdf_data** in the save method to pass the structure_to_save along with the data inside of it.
Note: the yellow highlighted functions are already implemented and must not be overridden.
The red highlighted functions must be implemented for every new experiment.
The blue highlighted functions are already implemented and may be overridden if needed.